พื้นฐานการจัดการข้อมูลอย่างเป็นระบบและปลอดภัย

Function 4.1 - The Secure Inventory Starter (CRUD)

🎯 วัตถุประสงค์

1. เพื่อให้นิสิตสามารถออกแบบและสร้าง **Mongoose Schema** พร้อมการทำ **Validation** ได้
2. เพื่อให้นิสิตเข้าใจการสร้าง API Endpoint ด้วย **Express** เพื่อติดต่อกับ **MongoDB**
3. เพื่อให้นิสิตฝึกการใช้ **Query Parameters** ในการกรองข้อมูล (Filtering)

🛠 ขั้นตอน

1. การเตรียมสภาพแวดล้อม

- สร้าง Folder ใหม่สำหรับโปรเจกต์ และรันคำสั่ง `npm init -y`
- ติดตั้ง Package ที่จำเป็น: `npm install express mongoose dotenv`
- สร้างไฟล์ `.env` และใส่ `MONGODB_URI` ของนิสิตให้เรียบร้อย

2. การออกแบบ Schema (Models/Product.js)

นิสิตต้องสร้าง Schema ที่ควบคุมกฎดังนี้:

- `name` : ต้องเป็น String, ห้ามซ้ำ (unique), และต้องกรอก (required)
- `price` : ต้องเป็น Number, ห้ามติดลบ (min: 0)
- `category` : ต้องเป็น String, บังคับเลือกได้แค่ 'Electronics', 'Home', 'Food' เท่านั้น
- `createdAt` : ให้บันทึกเวลาปัจจุบันโดยอัตโนมัติ

3. การสร้าง API (Controllers & Routes)

- **POST** `/api/products` : รับข้อมูลจาก Body และบันทึกลงฐานข้อมูล
- **GET** `/api/products` : ดึงข้อมูลทั้งหมด แต่ต้องเขียน Logic เพิ่มเพื่อเช็คว่าถ้ามีการส่ง `?minPrice=xxx` มา ให้ดึงเฉพาะสินค้าที่ราคา **มากกว่าหรือเท่ากับ** ค่านั้น

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

Checklist Function 4.1

C1: สามารถรัน Server และเชื่อมต่อ MongoDB Atlas ได้สำเร็จ

- **วิธีดู:** ดูที่ Terminal/Console ของ VS Code
- **เกณฑ์ผ่าน:** ต้องขึ้นข้อความ `Server running on port 5000` และตามด้วย `Connected to MongoDB` (ซึ่งมาจาก `.then()` ในโค้ด)

- **จุดที่ต้องระวัง:** หากขึ้น Error เกี่ยวกับ `IP Address` แสดงว่านิสิตลืมไป Config **Network Access** ใน MongoDB Atlas ให้เป็น `0.0.0.0/0`

C2: เมื่อ POST สินค้าที่ราคาติดลบ ระบบต้องตอบกลับด้วย Error (400)

- **วิธีดู:** ส่ง `POST` ไปที่ `/api/products` โดยใช้ Body: `{"name": "Test", "price": -50, "category": "Food"}`
- **เกณฑ์ผ่าน:** * Status Code ต้องเป็น **400 Bad Request** * Response Body ต้องมีข้อความ Error แจ้งว่า `Price cannot be less than 0` (หรือตามที่ตั้งไว้ใน Schema)
- **จุดที่ต้องระวัง:** หากระบบยอมให้เซฟค่า -50 ลงฐานข้อมูลได้ แสดงว่านิสิตลืมใส่ `min: 0` ใน Schema

C3: เมื่อ POST สินค้าที่มีหมวดหมู่ไม่อยู่ในกลุ่ม ระบบต้องตอบกลับด้วย Error

- **วิธีดู:** ส่ง `POST` โดยใช้ Body: `{"name": "Toy Car", "price": 100, "category": "Toy"}`
- **เกณฑ์ผ่าน:** * Status Code ต้องเป็น **400 Bad Request**
  - Error Message ต้องระบุว่าค่า `Toy` ไม่ได้รับอนุญาต (is not a valid enum value)
- **เกณฑ์เทคนิค:** นี่คือการเช็คว่านิสิตใช้ `enum: ['Electronics', 'Home', 'Food']` ถูกต้องไหม

C4: เมื่อ POST สินค้าชื่อซ้ำ ระบบต้องแจ้งเตือน (Unique Constraint)

- วิธีดู: 1. ส่ง POST ชื่อ "น้ำดื่ม" ครั้งแรก (ต้องผ่าน)

  2. ส่ง POST ชื่อ "น้ำดื่ม" อีกครั้งหนึ่ง
- **เกณฑ์ผ่าน:** การส่งครั้งที่ 2 ต้องได้ **400 Bad Request** และ Error มักจะระบุคำว่า `duplicate key error`
- **จุดที่ต้องระวัง:** หากผ่านทั้งสองครั้ง แสดงว่าลืมใส่ `unique: true` ที่ฟิลด์ `name` ใน Schema

C5: เมื่อเรียก GET /api/products?minPrice=500 ระบบแสดงผลถูกต้อง

- **วิธีดู:** สมมติใน DB มีสินค้าราคา 100, 500, และ 1000 ให้เรียก `GET /api/products?minPrice=500`
- **เกณฑ์ผ่าน:** ในรายการที่ตอบกลับมา (Array) ต้องมีสินค้าแค่ราคา 500 และ 1000 เท่านั้น (ตัวราคา 100 ต้องหายไป)
- **จุดที่ต้องระวัง:** ถ้าราคา 100 ยังโผล่มา แสดงว่า Logic ในการรับ `req.query.minPrice` ไปใส่ใน `Product.find()` ยังไม่ทำงาน

C6: มีฟิลด์ createdAt แสดงวันที่และเวลาถูกต้อง

- **วิธีดู:** ดูจาก Response ของคำสั่ง `GET` หรือ `POST` ที่สำเร็จ
- **เกณฑ์ผ่าน:** ต้องมีฟิลด์ `createdAt` และ `updatedAt` ปรากฏขึ้นมาพร้อมกับข้อมูล (เช่น `"createdAt": "2024-05-20T..."`)
- **เกณฑ์เทคนิค:** ตรวจสอบว่านิสิตใส่ `{ timestamps: true }` ไว้ในตำแหน่งที่ถูกต้องของ Schema (หลังปีกกาปิดของ Field Definitions)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

💡 **ข้อแนะนำในการตรวจ**

Capture หน้าจอ Postman ที่แสดงผลลัพธ์ทั้งตอนที่ **"ผ่าน (200/201)"** และตอนที่ **"ติด Error (400)"** ตามเงื่อนไขด้านบน เพื่อยืนยันว่านิสิตได้ทำ Validation ครบทุกขั้นตอนจริง

💡 คำแนะนำเพิ่มเติม

- หากนิสิตติดปัญหาเรื่อง **Validation Error** ให้ลองเช็คดูว่าใน `Product.js` ได้สะกดคำใน `enum` หรือชื่อฟิลด์ถูกต้องตามที่ `req.body` ส่งมาหรือไม่
- การใช้ `req.query.minPrice` ข้อมูลที่รับมาจะเป็น **String** อย่าลืมแปลงเป็น **Number** ก่อนนำไปเปรียบเทียบใน MongoDB

มาตรฐานที่ใช้ในระบบจัดการข้อมูล การลบข้อมูลแบบถาวร (Hard Delete) และการลบแบบเก็บข้อมูลไว้ (Soft Delete)

Function 4.2 - Soft Delete & Activity Tracking

🎯 วัตถุประสงค์

1. เพื่อให้นิสิตเข้าใจแนวคิดการทำ **Soft Delete** เพื่อรักษา Integrity ของข้อมูล
2. เพื่อให้นิสิตฝึกการใช้คำสั่ง **Update** เพื่อเปลี่ยนสถานะแทนการลบจริง
3. เพื่อให้นิสิตฝึกการเขียน **Query** เพื่อคัดกรองข้อมูลตามเงื่อนไขสถานะ (Status Filtering)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

🛠 ขั้นตอนการทำ

1. การออกแบบ Schema (Models/User.js)

นิสิตต้องสร้าง Schema สำหรับสมาชิก (Users) โดยมีฟิลด์บังคับดังนี้:

- `username` : String, required
- `email` : String, required
- `isDeleted` : **Boolean**, ต้องตั้งค่าเริ่มต้น (default) เป็น **false**
- `deletedAt` : **Date**, ให้เป็นค่า null ในตอนเริ่มต้น (จะใส่ค่าเมื่อถูกลบ)

2. การสร้าง API (Controllers & Routes)

- **POST** `/api/users` : สำหรับลงทะเบียนสมาชิกใหม่ (ค่า `isDeleted` ต้องเป็น false เสมอ)
- **GET** `/api/users` : ดึงรายชื่อสมาชิก **เฉพาะคนที่ยังไม่ถูกลบ** เท่านั้น (`isDeleted: false`)
- **DELETE** `/api/users/:id` : **(ส่วนสำคัญ)** แทนที่จะใช้ `findByIdAndDelete` ให้เปลี่ยนไปใช้การอัปเดตฟิลด์ `isDeleted` เป็น **true** และบันทึกเวลาที่ลบลงใน `deletedAt`

💻 Code

ไฟล์: `models/User.js`

ไฟล์: `server.js` **(โครงสร้างส่วน API)**

Checklist Function 4.2

C1: สถานะเริ่มต้นถูกต้อง (Default Value)

- **วิธีดู:** ให้นิสิตส่ง `POST /api/users` โดยส่งไปแค่ `username` และ `email` (ไม่ต้องส่งฟิลด์ `isDeleted` ไป)

- **เกณฑ์ผ่าน:** เมื่อดูผลลัพธ์ที่ตอบกลับมา หรือดูใน MongoDB Compass ต้องมีฟิลด์ `"isDeleted": false` ปรากฏขึ้นมาเองโดยอัตโนมัติ
- **จุดที่ต้องระวัง:** หากฟิลด์นี้ไม่ขึ้น หรือเป็น `null` แสดงว่านิสิตลืมใส่ `default: false` ใน Schema

C2: Soft Delete ทำงานจริง (Update instead of Delete)

- **วิธีดู:** ให้นิสิตยิง `DELETE /api/users/[ID_ของผู้ใช้]`
- **เกณฑ์ผ่าน:** เมื่อไปเช็คใน **MongoDB Compass** ข้อมูลของผู้ใช้ ID นั้น **"ต้องยังอยู่"** (ไม่หายไป) แต่ค่า `isDeleted` ต้องถูกเปลี่ยนจาก `false` เป็น `true`
- **จุดที่ต้องระวัง:** ถ้านิสิตเผลอใช้คำสั่ง `findByIdAndDelete` ข้อมูลจะหายไปเลย ซึ่งถือว่า **"ไม่ผ่าน"** เพราะผิดโจทย์เรื่องการรักษา Data Integrity

C3: บันทึกเวลาที่ลบ (deletedAt Tracking)

- **วิธีดู:** ดูข้อมูลใน MongoDB หลังจากการกด `DELETE`
- **เกณฑ์ผ่าน:** ฟิลด์ `deletedAt` ต้องเปลี่ยนจากเดิมที่เป็น `null` (หรือไม่มีอยู่) ให้กลายเป็นวันที่และเวลาปัจจุบัน (ISO Date)
- **เกณฑ์เทคนิค:** ตรวจสอบโค้ดใน Controller ว่านิสิตได้ใส่ `deletedAt: new Date()` ลงไปในคำสั่งอัปเดตหรือไม่

C4: การกรองข้อมูลสำเร็จ (GET Logic)

- วิธีดู: 1. เรียก GET /api/users (บันทึกไว้ว่ามีกี่คน)

  2. ทำการ DELETE ผู้ใช้ไป 1 คน

  3. เรียก GET /api/users ซ้ำอีกครั้ง
- **เกณฑ์ผ่าน:** จำนวนผู้ใช้ในครั้งที่ 2 **ต้องลดลง** และคนที่เราเพิ่งลบไปต้องไม่ปรากฏในรายชื่อ
- **จุดที่ต้องระวัง:** ถ้านิสิตยังเห็นคนเดิมโผล่มา แสดงว่าลืมใส่เงื่อนไข `{ isDeleted: false }` ในคำสั่ง `User.find()` ของ Endpoint GET

C5: Response สวยงาม (User Experience)

- **วิธีดู:** ดู JSON Response ที่ตอบกลับมาจากคำสั่ง `DELETE`
- **เกณฑ์ผ่าน:** ควรมีข้อความแจ้งเตือนที่ชัดเจน เช่น `{ "message": "ลบข้อมูลสำเร็จ", "user": { ... } }` และ Status Code ควรเป็น **200 OK**
- **เกณฑ์เทคนิค:** หากนิสิตลบ ID ที่ไม่มีอยู่จริง (เช่น มั่วเลข ID) ระบบควรตอบกลับเป็น **404 Not Found** ไม่ใช่ Error 500

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

💡 คำแนะนำ

- **"ถ้าอยากกู้คืน (Restore) ผู้ใช้คนนี้กลับมา ต้องทำอย่างไร?"** > *คำตอบที่ถูกต้องคือ:* แค่สร้าง Endpoint ใหม่ที่ `PATCH` ค่า `isDeleted` กลับไปเป็น `false` และแก้ `deletedAt` เป็น `null` ข้อมูลก็จะกลับมาแสดงผลที่หน้าเว็บทันทีโดยไม่ต้องกรอกข้อมูลใหม่
- **ทำไมต้องทำ Soft Delete?** ลองจินตนาการว่าถ้าสมาชิกคนนี้เคยสั่งซื้อสินค้าไปแล้ว (Orders) หากเราลบข้อมูลสมาชิกทิ้งถาวร ข้อมูลการสั่งซื้อในอดีตจะหาชื่อผู้ซื้อไม่เจอทันที (Data Integrity พัง) การทำ Soft Delete จึงช่วยรักษาความสัมพันธ์ของข้อมูลไว้ได้
- **Advanced Tip:** หากนิสิตทำเสร็จเร็ว ลองสร้าง Endpoint `GET /api/users/trash` เพื่อดูเฉพาะคนที่ถูกลบไปแล้ว

จัดการความสัมพันธ์ระหว่าง 2 Collection เขียน Logic ตรวจสอบเงื่อนไขทางธุรกิจ (Business Logic) ทำ Atomic Update เพื่อป้องกันข้อมูลผิดพลาด การจัดการ Transaction เบื้องต้น, การทำ Atomic Update และ Logic การตัดสต็อก

Function 4.3 - The Smart Inventory - Atomic Stock Management

🎯 วัตถุประสงค์

1. เพื่อให้นิสิตฝึกการจัดการข้อมูลที่มีความเกี่ยวข้องกัน (Cross-collection interaction)
2. เพื่อให้นิสิตเข้าใจการใช้ **Conditional Updates** เพื่อป้องกันการขายสินค้าเกินสต็อก (Overselling)
3. เพื่อให้นิสิตฝึกการเขียน Logic ตรวจสอบเงื่อนไขก่อนการบันทึกข้อมูล (Validation Logic)

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

🛠 ขั้นตอน

1. การเตรียม Database

นิสิตต้องมี 2 Collection ดังนี้:

- **Products:** เก็บข้อมูล `name` , `price` , และ `stock` (Number, ห้ามติดลบ)
- **Orders:** เก็บข้อมูล `productId` , `quantity` (จำนวนที่ซื้อ), และ `totalPrice`

2. การสร้าง API: POST `/api/orders`

เมื่อมีการสั่งซื้อเข้ามา นิสิตต้องเขียน Code ให้ทำงานตามลำดับดังนี้:

1. **Find:** ค้นหาตัวสินค้าจาก `productId` ที่ส่งมา
2. **Check:** ตรวจสอบว่าสินค้ามีอยู่จริง และ `stock` ในฐานข้อมูลมีเพียงพอกับ `quantity` ที่สั่งซื้อหรือไม่
3. **Atomic Update:** หากสต็อกพอ ให้ทำการลดค่า `stock` ใน `Products` และบันทึกข้อมูลลงใน `Orders`
4. **Error Handling:** หากสต็อกไม่พอ ต้องไม่เกิดการบันทึกใดๆ และตอบกลับ Error ที่เหมาะสม

💻 Code Starter Template

ไฟล์: `models/Product.js`

ไฟล์: `server.js` **(ส่วนของ Logic การสั่งซื้อ)**

Checklist Function 4.3

C1: การเช็คสต็อกก่อนซื้อ (Pre-check Validation)

- วิธีดู: 1. ตรวจสอบใน Database ว่าสินค้า A มี stock: 5

  2. ใช้ Postman ส่ง POST /api/orders โดยระบุ quantity: 10

- **เกณฑ์ผ่าน:** * ต้องได้รับ **Status 400 Bad Request**
  - ในฐานข้อมูลต้อง **ไม่มี** Order ใหม่เกิดขึ้น และ Stock ของสินค้า A ต้องยังคงเป็น **5** เท่าเดิม
- **จุดที่ต้องระวัง:** หากระบบยอมให้สั่งซื้อได้จนสต็อกติดลบ หรือสร้าง Order สำเร็จทั้งที่ของไม่พอ แสดงว่า Logic การ Check ล้มเหลว

C2: ความถูกต้องของสต็อก (Stock Deduction)

- วิธีดู: 1. ดูค่าสต็อกก่อนซื้อ (สมมติมี 100)

  2. สั่งซื้อสินค้าจำนวน 3 ชิ้น (quantity: 3)

  3. กลับไปดูค่าสต็อกใน MongoDB Compass

- **เกณฑ์ผ่าน:** ค่าสต็อกใน Database ต้องลดลงเหลือ **97** ทันทีหลังจากได้รับ Status 201
- **จุดที่ต้องระวัง:** นิสิตบางคนอาจจะลดค่าสต็อกใน Code (JavaScript) แล้ว `save()` ทับ ซึ่งอาจเกิดปัญหาถ้ามีคนสั่งพร้อมกัน

C3: ความถูกต้องของราคา (Server-side Calculation)

- **วิธีดู:** ตรวจสอบข้อมูลใน Collection `Orders` หลังสั่งซื้อสำเร็จ
- **เกณฑ์ผ่าน:** ฟิลด์ `totalPrice` ต้องเท่ากับ `ราคาต่อชิ้น x จำนวนที่สั่ง`
- **จุดที่ต้องระวัง: ห้ามให้นิสิตรับค่า totalPrice มาจาก Body (Frontend)** เพราะ User สามารถแก้ไขราคาเองได้ใน Postman (เช่น แก้ราคาจาก 1,000 เหลือ 1 บาท) ดังนั้น Server ต้องเป็นคนคำนวณเองเท่านั้น

C4: Atomic Operation (Concurrency Protection)

- **วิธีดู:** ตรวจดูในไฟล์ Controller ของนิสิต
- **เกณฑ์ผ่าน:** ต้องมีการใช้คำสั่ง `$inc` ร่วมกับค่าติดลบ (เช่น `stock: -quantity`) ภายใน `findOneAndUpdate` หรือ `updateOne`
- **เหตุผล:** วิธีนี้คือการสั่งให้ MongoDB เป็นคนลบเลขเองที่ระดับ Engine ซึ่งจะปลอดภัยกว่าการดึงตัวเลขออกมาลบใน Node.js (เพราะถ้ามี 2 คำขอเข้ามาพร้อมกันในเสี้ยววินาที การลบใน Node.js จะทำให้สต็อกเพี้ยนได้)

C5: Data Integrity (Error Recovery)

- **วิธีดู:** ให้นิสิตลอง "แกล้งทำระบบพัง" เช่น ลองลบ Schema ของ Order ทิ้งชั่วคราวเพื่อให้ตอน `save()` Order เกิด Error
- **เกณฑ์ผ่าน:** หากการสร้าง Order ล้มเหลว **สต็อกสินค้าต้องไม่ถูกตัดออกไปฟรีๆ** * **เกณฑ์เทคนิค:** หากนิสิตใช้เทคนิคขั้นสูงอย่าง **Mongoose Transactions (`session`)** จะถือว่าทำได้ดีเยี่ยมมาก เพราะหากส่วนใดส่วนหนึ่งพัง ระบบจะ Rollback (คืนค่า) ทั้งหมดอัตโนมัติ

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

💡 สรุปการตรวจ

ควรดู **"ลำดับการทำงาน"** เป็นหลัก

1. **ถามหาของ** (Find Product)
2. **เช็คว่าพอไหม** (Check Stock)
3. **ตัดของ + บันทึกการขาย** (Deduct Stock & Create Order)

หากข้ามขั้นตอนใดขั้นตอนหนึ่ง หรือทำสลับลำดับ (เช่น ตัดสต็อกไปก่อนแล้วค่อยเช็ค) อาจทำให้ข้อมูลในระบบจริงพังได้ในอนาคต

การค้นหาข้อมูลขั้นสูง (Advanced Query) การจัดการข้อมูลประเภท Array และการทำ Pagination การใช้ MongoDB Operators ($all), การจัดการ Array และ Pagination

---

Function 4.4 - Multi-Tag Search & Pagination

🎯 วัตถุประสงค์

1. เพื่อให้นิสิตสามารถใช้ MongoDB Query Operators ขั้นสูง เช่น `$all` หรือ `$in` ได้
2. เพื่อให้นิสิตเข้าใจการจัดการข้อมูลประเภท **Array** ภายใน Document
3. เพื่อให้นิสิตสามารถเขียนระบบ **Pagination** (limit, skip) และคำนวณข้อมูลหน้า (Metadata) ได้

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

🛠 ขั้นตอนการทำ

1. การออกแบบ Schema (Models/Post.js)

สร้าง Schema สำหรับโพสต์เนื้อหา:

- `title` : String, required
- `content` : String, required
- `tags` : **[String]** (Array ของ String เช่น `["javascript", "node", "web"]`)
- `createdAt` : Date (อัตโนมัติ)

2. การสร้าง API: GET `/api/posts/search`

นิสิตต้องสร้าง Endpoint ที่รองรับ Query Parameters ดังนี้:

- `tags` : เช่น `?tags=node,express` (ต้องคืนค่าโพสต์ที่มีแท็ก **ครบทุกตัว** ที่ส่งมา)
- `page` : หน้าที่ต้องการเรียก (Default: 1)
- `limit` : จำนวนข้อมูลต่อหน้า (Default: 5)

💻 Code

ไฟล์: `server.js` (Logic การค้นหาและแบ่งหน้า)

Checklist Function 4.4

C1: Exact Tag Matching (การดึงข้อมูลแบบตรงเงื่อนไขทั้งหมด)

- **วิธีดู:** ให้นิสิตสร้างโพสต์ที่มีแท็กต่างกัน เช่น:
  1. Post A: `["node"]`

2. Post B: `["node", "express"]`
3. Post C: `["express"]`

- **การทดสอบ:** เรียก `GET /api/posts/search?tags=node,express`
- **เกณฑ์ผ่าน:** ต้องแสดง **เฉพาะ Post B เท่านั้น**
- **จุดที่ต้องระวัง:** หาก Post A หรือ C โผล่มา แสดงว่านิสิตใช้ `$in` (มีอย่างใดอย่างหนึ่ง) แทนที่จะใช้ **`$all`** (ต้องมีครบ) ซึ่งถือว่าผิดวัตถุประสงค์ของ Multi-tag Search

C2: Array Parsing (การแปลงข้อมูลจาก URL)

- **วิธีดู:** ดูในโค้ดส่วนที่รับค่าจาก `req.query.tags`
- **เกณฑ์ผ่าน:** ต้องมีการใช้คำสั่ง **`.split(',')`** หรือฟังก์ชันอื่นที่แปลง String `"node,express"` ให้กลายเป็น Array `["node", "express"]` ก่อนส่งเข้าไปใน Query ของ Mongoose
- **เหตุผล:** MongoDB จะไม่สามารถค้นหาด้วย Operator `$all` ได้หากเราส่งไปเป็น String ก้อนเดียว

C3: Skip & Limit Logic (ความถูกต้องของการแบ่งหน้า)

- **วิธีดู:** สมมติมีข้อมูล 10 รายการ และตั้งค่า `?limit=3`
- **การทดสอบ:** เรียก `page=1` (ได้ 3 รายการแรก) -> เรียก `page=2` (ได้ 3 รายการถัดไป)
- **เกณฑ์ผ่าน:** ข้อมูลในหน้า 1 และหน้า 2 **ต้องไม่ซ้ำกัน** และลำดับต่อเนื่องกันตามเวลาที่บันทึก
- **เกณฑ์เทคนิค:** เช็คสูตรการคำนวณ `skip = (page - 1) * limit` ในโค้ดว่าถูกต้องหรือไม่

C4: Pagination Metadata (ข้อมูลสรุปภาพรวม)

- **วิธีดู:** ดูที่ JSON Response ในส่วนของฟิลด์ `metadata`
- **เกณฑ์ผ่าน:** * `totalPosts` : ต้องตรงกับจำนวนข้อมูลทั้งหมดใน Database ที่ตรงเงื่อนไข
  - `totalPages` : ต้องคำนวณถูก เช่น ถ้ามี 11 รายการ และ limit คือ 5 ต้องได้ `totalPages: 3`
- **จุดที่ต้องระวัง:** นิสิตมักลืมใช้ `Math.ceil()` ทำให้ได้ผลลัพธ์เป็นทศนิยม ซึ่งหน้าเว็บจะนำไปใช้ต่อไม่ได้

C5: Empty Result Handling (การจัดการกรณีไม่พบข้อมูล)

- **วิธีดู:** ค้นหาด้วยแท็กที่ไม่มีอยู่จริง เช่น `?tags=superman`
- **เกณฑ์ผ่าน:** ระบบต้องตอบกลับด้วย Status **200 OK** และ `data: []` (Array ว่าง)
- **จุดที่ต้องระวัง:** ห้ามตอบกลับด้วย Error 500 หรือ Error 404 เพราะการ "หาไม่เจอ" ในฟีเจอร์ Search ถือเป็นพฤติกรรมปกติของระบบ ไม่ใช่ความผิดพลาดของโปรแกรม

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

การใช้ฟีเจอร์ของ Mongoose คือ **Virtuals** (ฟิลด์เสมือนที่ไม่ได้เก็บใน DB) และ **Aggregation Pipeline** (การประมวลผลข้อมูลชุดใหญ่แบบซับซ้อน) เพื่อดึงประสิทธิภาพของ MongoDB ออกมาให้สูงสุด หัวข้อ: Mongoose Virtuals และ MongoDB Aggregation Framework

---

Function 4.5 - The Smart Discount Engine

🎯 วัตถุประสงค์

1. เพื่อให้นิสิตสามารถสร้าง **Virtual Fields** เพื่อคำนวณค่าจากข้อมูลที่มีอยู่โดยไม่เพิ่มภาระให้พื้นที่จัดเก็บ
2. เพื่อให้นิสิตเข้าใจและใช้งาน **Aggregation Pipeline** (`$group` , `$avg` , `$max`) เพื่อทำรายงานสรุปข้อมูล
3. เพื่อให้นิสิตฝึกการจัดการข้อมูลเชิงลึก (Data Insight) ในระดับ Database

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

🛠 ขั้นตอนการทำ

1. การออกแบบ Schema (Models/Product.js)

สร้าง Schema สินค้าที่มีระบบส่วนลด:

- `name` : String, required
- `category` : String (Electronics, Home, Fashion)
- `originalPrice` : Number, required (ราคาตั้งต้น)
- `discountPercent` : Number, default: 0 (เปอร์เซ็นต์ส่วนลด 0-100)

2. ความท้าทายที่ 1: การใช้ Virtuals

สร้างฟิลด์เสมือนชื่อ **`salePrice`** โดยมีสูตรคือ: `originalPrice * ((100 - discountPercent) / 100)`

- *เงื่อนไข:* ฟิลด์นี้ต้องปรากฏเมื่อส่งข้อมูลออกไปทาง JSON (API) แต่ **ห้าม** ปรากฏในฐานข้อมูล MongoDB Compass

3. ความท้าทายที่ 2: การทำ Aggregation Report

สร้าง Endpoint: `GET /api/products/stats` เพื่อคำนวณค่าดังนี้:

- ราคาสูงสุดหลังหักส่วนลด (`maxSalePrice`)
- ราคาเฉลี่ยหลังหักส่วนลดทั้งหมดในร้าน (`avgSalePrice`)
- จำนวนสินค้าแยกตามหมวดหมู่ (`totalByPage`)

💻 Code Starter Template

ไฟล์: `models/Product.js` (การตั้งค่า Virtuals)

ไฟล์: `server.js` (ส่วนของ Aggregation)

Checklist Function 4.5

ข้อ C1: Virtual Field Display (การแสดงผลฟิลด์เสมือน)

- **วิธีดู:** ให้นิสิตยิง `GET /api/products`
- **เกณฑ์ผ่าน:** ในข้อมูลสินค้าแต่ละชิ้น ต้องมีฟิลด์ `salePrice` โผล่มาใน JSON โดยที่ค่าของมันต้องตรงกับ `originalPrice` หักลบด้วย `discountPercent`
- **การทดสอบ:** ลองแก้ `originalPrice` ใน Database แล้วยิง API ใหม่ ค่า `salePrice` ต้องเปลี่ยนตามทันทีโดยไม่ต้องไปสั่งบันทึกใหม่

ข้อ C2: No Redundant Data (การตรวจสอบความสะอาดของ DB)

- **วิธีดู:** อาจารย์ต้องให้เขาสกรีนหน้าจอ **MongoDB Compass** หรือ **Mongo Shell**
- **เกณฑ์ผ่าน:** ใน Document ของสินค้า **"ห้าม"** มีฟิลด์ที่ชื่อ `salePrice` ปรากฏอยู่จริง
- **เหตุผล:** ถ้ามีแสดงว่านิสิตแอบไปบันทึกลงไปในฐานข้อมูล ซึ่งผิดหลักการ Virtuals ที่ต้องการลดการเก็บข้อมูลซ้ำซ้อน (Redundant Data)

ข้อ C3: Aggregation Accuracy (ความแม่นยำของรายงาน)

- **วิธีดู:** ให้นิสิตยิง `GET /api/products/stats`
- **เกณฑ์ผ่าน:** ค่า `avgSalePrice` และ `maxSalePrice` ต้องคำนวณมาจากราคาที่หักส่วนลดแล้วเท่านั้น
- **จุดที่ต้องระวัง:** นิสิตที่ทำไม่ผ่าน มักจะใช้ราคาเต็ม (`originalPrice`) มาหาค่าเฉลี่ย ซึ่งจะทำให้รายงานตัวเลขผิดพลาดในเชิงธุรกิจ

ข้อ C4: Schema Configuration (การตั้งค่า Schema)

- **วิธีดู:** ตรวจสอบโค้ดในไฟล์ `models/Product.js`
- **เกณฑ์ผ่าน:** ในการประกาศ `new mongoose.Schema({ ... }, { options })` ต้องมีการใส่ `toJSON: { virtuals: true }`
- **เหตุผล:** หากไม่ใส่ Option นี้ แม้นิสิตจะเขียนโค้ด Virtual ไว้ถูกต้อง แต่มันจะไม่ถูก "Serialise" ออกมาเป็น JSON เมื่อส่งข้อมูลผ่าน API ทำให้หน้าบ้านมองไม่เห็นฟิลด์นี้

ข้อ C5: Empty State Handling (การจัดการกรณีไม่มีข้อมูล)

- **วิธีดู:** ลบสินค้าใน Database ออกให้หมด แล้วยิงเรียก API สถิติ
- **เกณฑ์ผ่าน:** ระบบต้องตอบกลับเป็น `null` หรือ Object เปล่าที่มีค่าเป็น 0
- **จุดที่ต้องระวัง:** หาก Server ค้างหรือตอบกลับด้วย **500 Internal Server Error** แสดงว่านิสิตไม่ได้เตรียมรับมือกรณี `stats[0]` เป็น `undefined` (เพราะ Aggregation จะคืนค่าเป็น Array ว่างถ้าไม่มีข้อมูล)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ